FSCT 8561 — Security Applications

MIDTERM EXAM | OPERATION SENTINEL — SECURE SERVER ASSESSMENT

Instructor: Dr. Maryam R. Aliabadi | Duration: 72 hours (3 days)

Format: Individual, open-book practical exam | Coverage: Labs 0–5

Scenario

You have been hired as a cybersecurity analyst to develop a Python-based security assessment application for a small organization. The organization wants to identify exposed TCP services, assess the security configuration of a web application, and investigate suspicious network traffic. It also requires an authenticated administrative interface for internal diagnostic information.

Design, implement, test, and evaluate a complete client–server security application using the techniques introduced in Labs 0–5. Your submission must be a working, integrated application—not a collection of unrelated lab scripts.

Instructor-Provided Resources

- FSCT8561_Operation_Sentinel_Traffic.pcap — synthetic traffic for offline analysis.
- test_website.py — local HTTP test service for security configuration checks.
- README_Students.txt — instructions for running the test environment.

Students develop their own TCP client and TCP server. No public website, virtual machine, or external deployment is required.

Required System Architecture

CLIENT — Analyst Workstation	SERVER — Student-Built TCP Service
TCP client: connects, sends commands, displays responses	TCP server: accepts connections and processes commands
Port scanner (Lab 2): checks TCP ports on the student-built server	Authentication (Lab 3): verifies password and TOTP MFA
HTTP checker (Lab 5): checks the supplied local HTTP service	PCAP analyzer (Lab 4): analyzes supplied file using Scapy
User interface: login, menu, results	Authorization, validation, security event logging

Communication: Client ⇄ TCP socket ⇄ Server (Labs 0–1). The client requests PCAP analysis through the authenticated server connection; the server returns results. Client-side scanning and HTTP checks run locally. Scan only authorized localhost/test targets.

Day 1 — Architecture and Threat Modeling (25%)

Part A — Architecture Design (10%)

Create your own system architecture diagram showing the client, server, TCP communication channel, and security modules. Explain component responsibilities, information flows, and the rationale for your stateful or stateless interaction model. The reference architecture above specifies component placement; your diagram must detail your implementation.

Part B — Adversarial Threat Model (15%)

Identify at least five distinct threats covering sockets, scanning, authentication/MFA, Scapy-based analysis, and HTTP assessment. For each threat, provide the asset, attack vector, impact, mitigation, and any residual risk. Use a structured table.

Day 2 — Protocol and Component Specification (25%)

Part C — Protocol Design (15%)

Specify the TCP application-layer protocol: message structure and command format; authentication and MFA sequence; authorization rules for protected commands; malformed request handling; error responses; session termination; and replay/security limitations. Clearly distinguish protections implemented from those proposed for future deployment.

Part D — Component Design (10%)

For the port scanner, MFA module, HTTP checker, and PCAP analyzer, specify inputs, outputs, expected behavior, failure conditions, and security controls. Identify one category of sensitive information you intentionally avoid collecting or logging and justify the choice.

Day 3 — Implementation, Testing, and Analysis (50%)

Part E — Full Integrated Implementation (30%)

Build a fully functional Python client–server application. It must demonstrate:

1. TCP connection establishment, framed or otherwise unambiguous message exchange, and command processing.
2. Password verification using a secure password-hashing approach and TOTP-based MFA; protect authenticated server operations.
3. Rejection of unauthenticated or unauthorized requests to protected server functions.
4. Client-side TCP port scanning of an instructor-authorized range that includes the student server's listening port.
5. Client-side HTTP checks using requests against the supplied local test website.
6. Server-side offline PCAP analysis using Scapy, including traffic summary and at least one justified suspicious-pattern detection.
7. Transmission of PCAP analysis results from server to client.
8. Input validation, meaningful error handling, and security-event logging without recording passwords or OTPs.

Use the techniques and libraries covered in Labs 0–5 (including socket, hashlib, pyotp, Scapy, requests, and python-nmap where applicable), plus standard Python utilities needed to organize the program. Comment on why key security controls exist. Pseudocode, isolated lab scripts, or a minimal core-logic demonstration are insufficient.

Part F — Comprehensive Testing (15%)

Execute and document at least ten tests, covering successful/failed authentication, unauthorized command rejection, an open and a closed TCP port, HTTP security findings and connection failure, PCAP summary and detection, invalid inputs, and server disconnection. For every test, provide expected result, actual result, pass/fail status, and evidence (screenshots or captured output).

Part G — Security Analysis and Reflection (5%)

9. Which threats from your Day 1 model did your implementation mitigate, and which remain?
10. What suspicious traffic did your PCAP analyzer identify? Cite packet-level or summary evidence.
11. What did your HTTP checker find, and why are its findings not proof of exploitability?
12. What are the confidentiality and replay limitations of a plain TCP connection, even with TOTP MFA?

13. What would you improve before deploying this application in production?

Scope, Safety, and Security Assumptions

Restrict port scanning and HTTP checks to the provided local test environment or other explicitly authorized targets. Do not scan public systems or third-party websites. Do not exploit vulnerabilities, generate malicious traffic, or deploy public-facing services. A basic TCP/TOTP implementation is not encrypted by default: do not claim confidentiality against sniffing without implementing an appropriate transport-security mechanism. Discuss that limitation in Part G.

Submission Requirements

- One PDF containing architecture, threat model, protocol and component specifications, test evidence, and security analysis.
- A GitHub repository link containing all Python source files, dependencies, and a README with setup, run, and testing instructions.
- PDF filename: FSCT8561-Midterm-FirstName-LastName-StudentNumber.pdf
- Deadline: 72 hours after exam release.

Grading Summary

Component	Weight
A — Architecture design	10%
B — Threat modeling	15%
C — Protocol design	15%
D — Component design	10%
E — Full implementation	30%
F — Testing	15%
G — Security analysis	5%
TOTAL	100%

AI Policy & Academic Integrity

This is an individual, open-resource assessment. You may consult documentation and AI tools as reference resources, subject to course academic integrity requirements. Your submitted architecture, code, testing evidence, and written explanations must represent work you understand and can independently defend. Design, implementation, testing, and analysis must be internally consistent.